

XXVI SIMPÓSIO BRASILEIRO DE CIBERSEGURANÇA · SBSEG 2026

1 a 4 de setembro de 2026 · Armação dos Búzios, RJ

AlaCGateGuard: A Security-First Pipeline for Benchmarking LLM- and SLM-Generated IaC



Francis Luis Santos Vargas, Rodrigo Brandão Mansilha, Diego Kreutz

AI Horizon Labs · Universidade Federal do Pampa (UNIPAMPA)
Programa de Pós-Graduação em Engenharia de Software (PPGES)

O problema

laC acelera a implantação, mas também escala configurações inseguras e exposições que podem permanecer meses sem detecção.

23%

dos incidentes de nuvem vêm de recursos mal configurados



Threat #1

Top Threat #1 — Misconfiguration



40%

De código vulnerável ainda é gerado por IA em cenários sensíveis



O que é difícil na automação

- Como medir a qualidade do resultados final
 - Checkov mede conformidade de políticas;
 - Trivy mede severidade de vulnerabilidades.
- Modelos variam em custo e latência.
 - LLMs via API
 - SLMs locais: desafio adicional: qualidade dos resultados

O que já existe e o que falta

Abordagem	Foco	Pipeline
laC-Eval [Kon et al.]	Código válido	Segmentado
DPlac-Eval [Zhang et al.]	Código válido	Segmentado
AlaCGateGuard (nosso)	Código Seguro	Integrado e automatizado

laC-Eval e DPlac-Eval avaliam sintaxe/deployability; scanners manuais não escalam entre modelos e prompts.

01

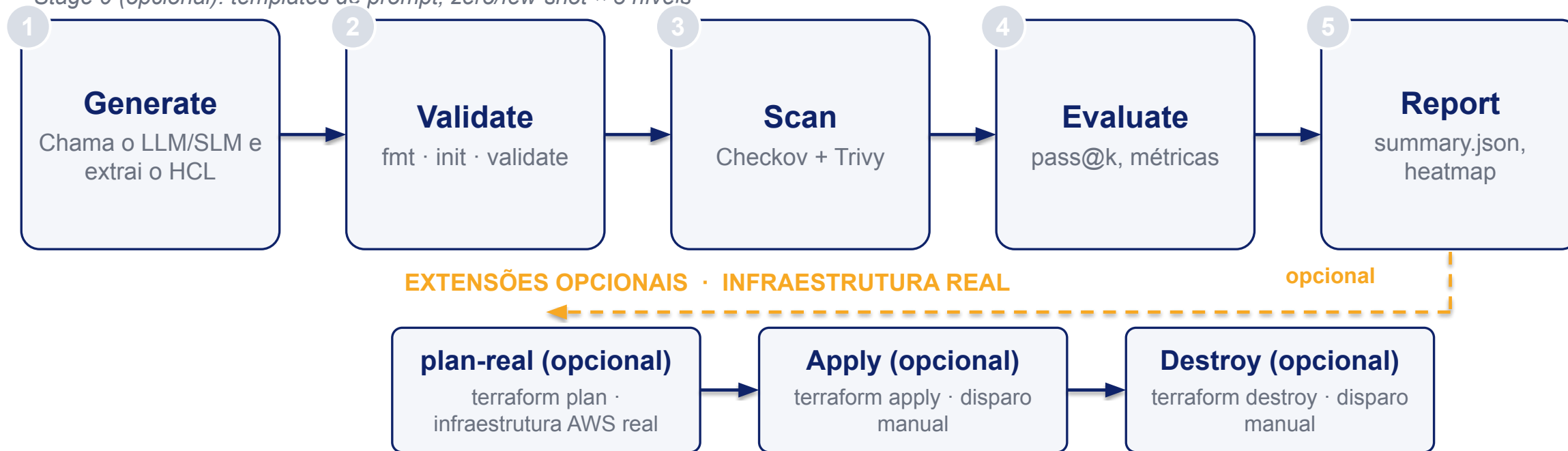
Solução proposta

A contribuição central do trabalho

Arquitetura macro do pipeline

PIPELINE PRINCIPAL · 5 ESTÁGIOS, SEMPRE EXECUTADOS

Stage 0 (opcional): templates de prompt, zero/few-shot × 3 níveis



↳ também acionável de forma conversacional via servidor MCP

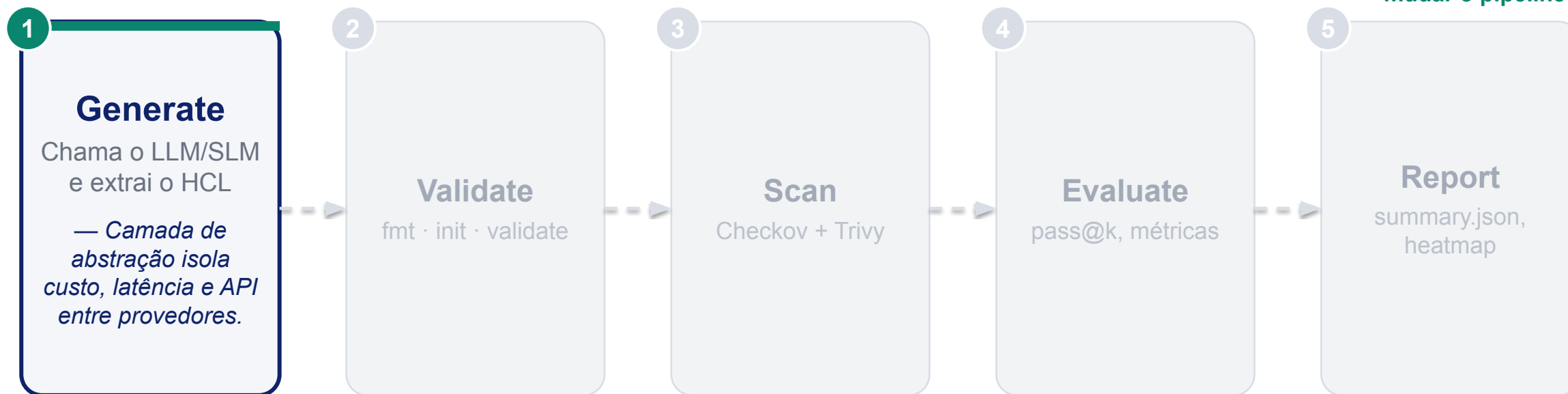
Um único fluxo: 5 estágios centrais sempre executados, mais duas extensões opcionais, infraestrutura real e interface conversacional via MCP.

Estágio 1 de 5 — Generate

Generate → Validate → Scan → Evaluate → Report | Infra real → MCP

Stage 0 (opcional): templates de prompt, zero/few-shot × 3 níveis

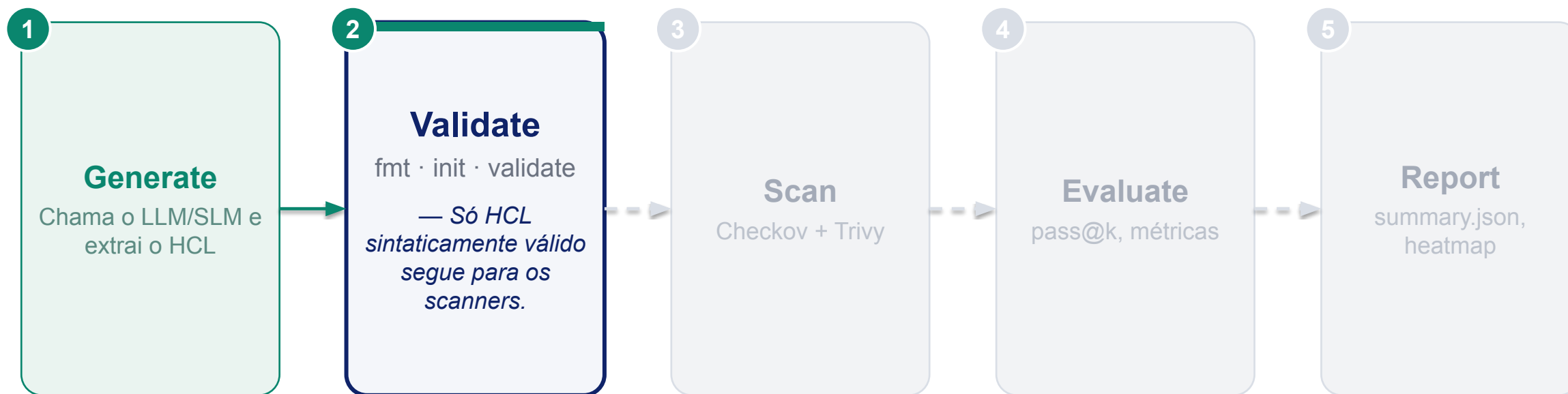
1 arquivo YAML → novo modelo, cenário ou scanner, sem mudar o pipeline



Estágio 1 — o pipeline chama o LLM ou SLM com o prompt do cenário e extrai o bloco de código HCL gerado.

Estágio 2 de 5 — Validate

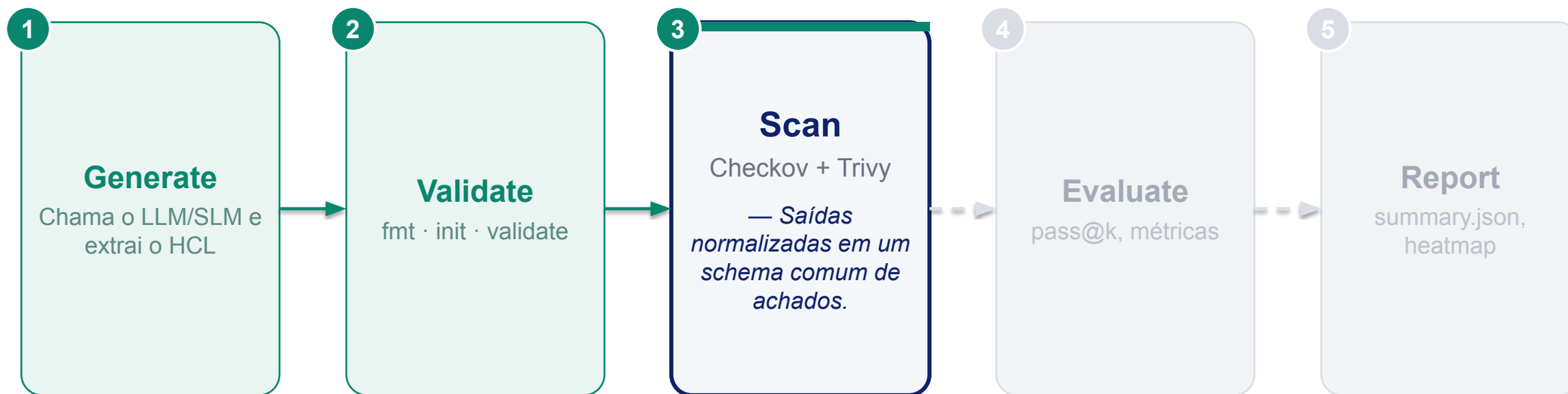
Generate → **Validate** → Scan → Evaluate → Report | Infra real → MCP



Estágio 2 — terraform fmt, init e validate confirmam que o HCL gerado é sintaticamente correto.

Estágio 3 de 5 — Scan

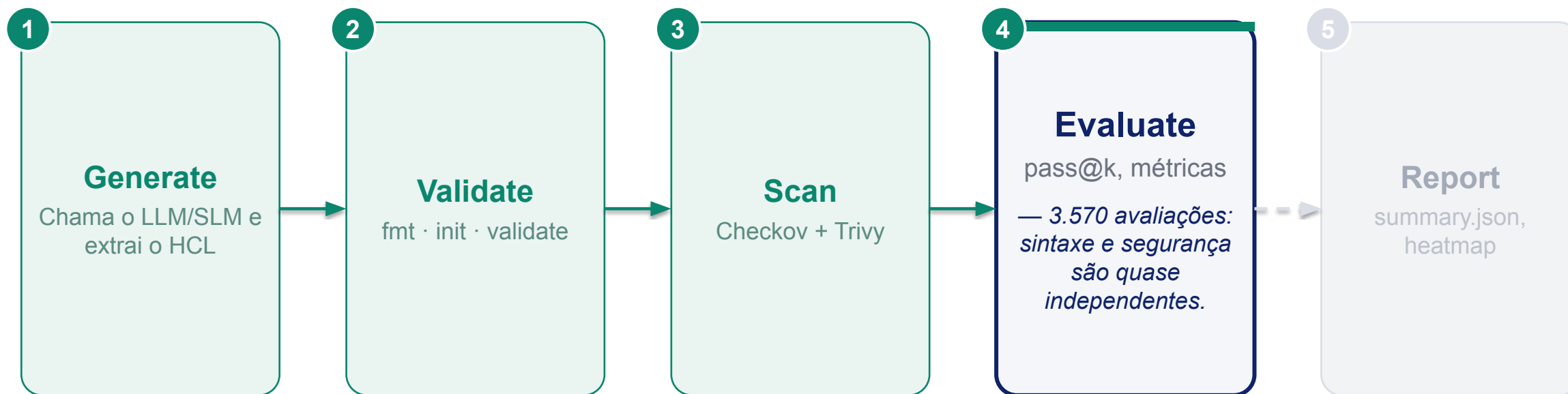
Generate → Validate → **Scan** → Evaluate → Report | Infra real → MCP



Estágio 3 — Checkov e Trivy analisam o HCL validado; as saídas são normalizadas em um schema comum.

Estágio 4 de 5 — Evaluate

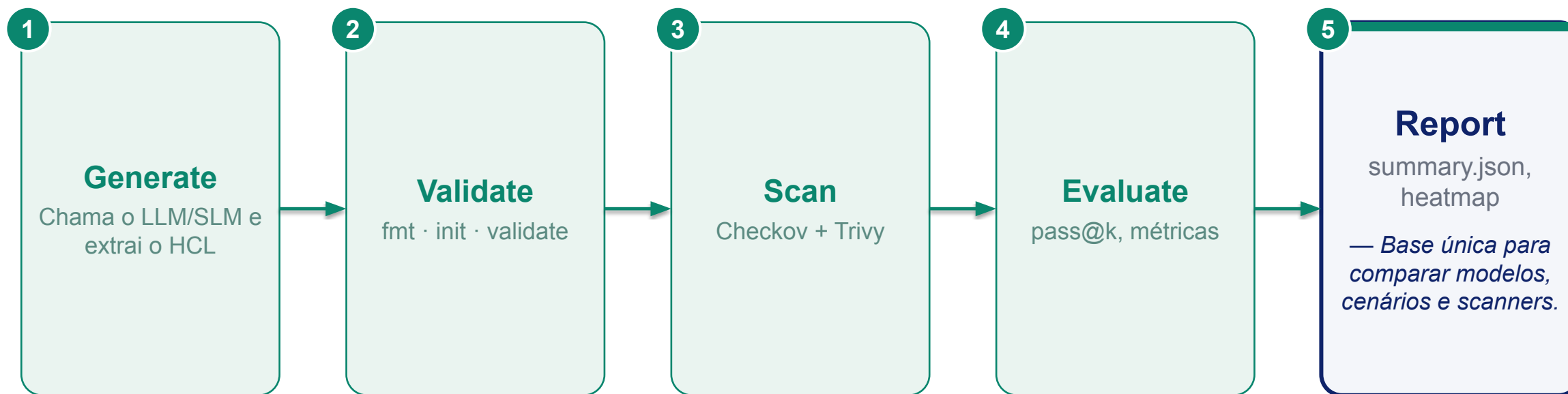
Generate → Validate → Scan → **Evaluate** → Report | Infra real → MCP



Estágio 4 — $pass@k$ e métricas de conformidade de segurança são calculadas a partir dos achados dos scanners.

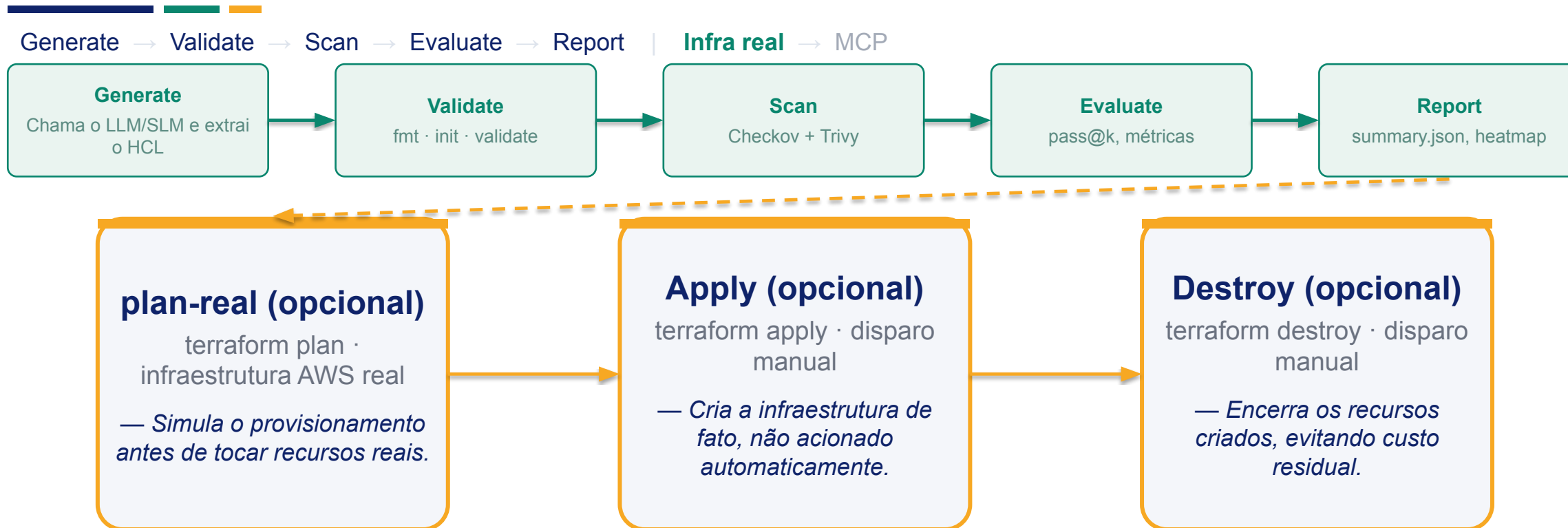
Estágio 5 de 5 — Report

Generate → Validate → Scan → Evaluate → **Report** | Infra real → MCP



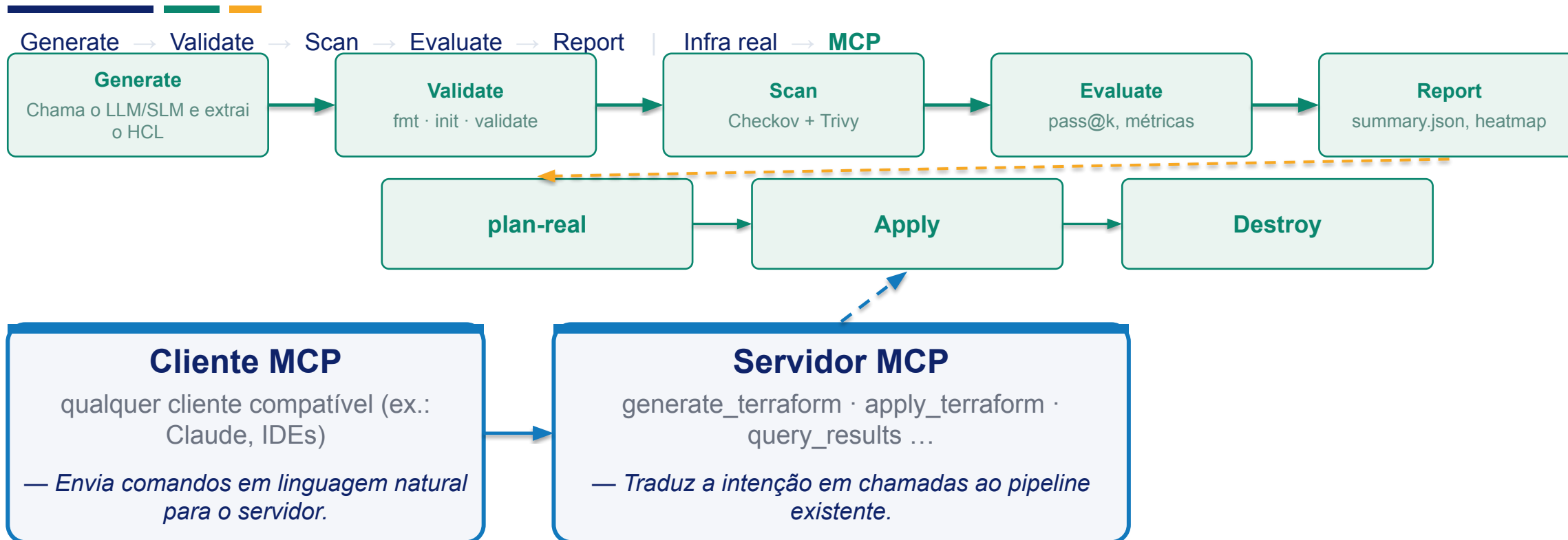
Estágio 5 — tudo é consolidado em summary.json e heatmap, prontos para comparar modelos e cenários.

Extensão opcional — Infraestrutura real



Depois do Report, o pipeline pode opcionalmente provisionar infraestrutura AWS real, sempre com disparo manual.

Extensão opcional — Interface conversacional (MCP)

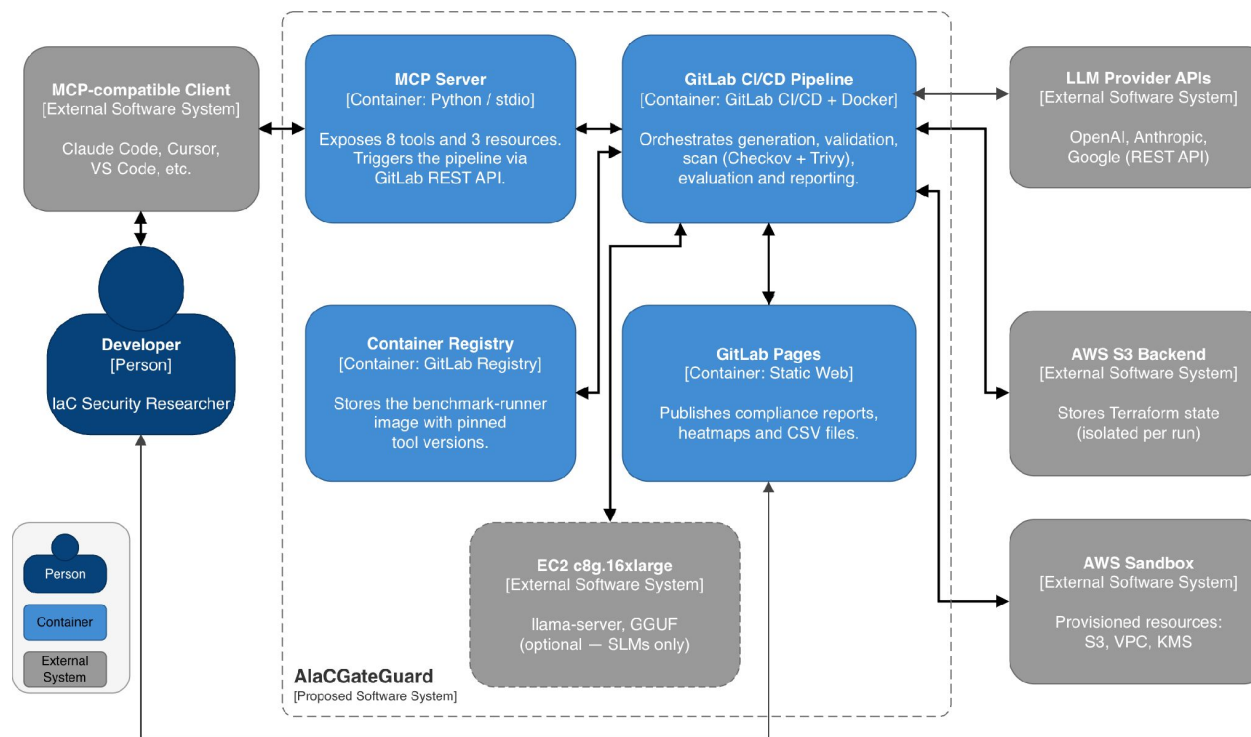


Qualquer cliente MCP aciona o pipeline por linguagem natural; o servidor expõe `generate_terraform`, `apply_terraform` e `query_results`.

01 · SOLUÇÃO PROPOSTA

Arquitetura de implantação da ferramenta

Visão geral → Aciona a geração → Guarda os artefatos → Extensões opcionais

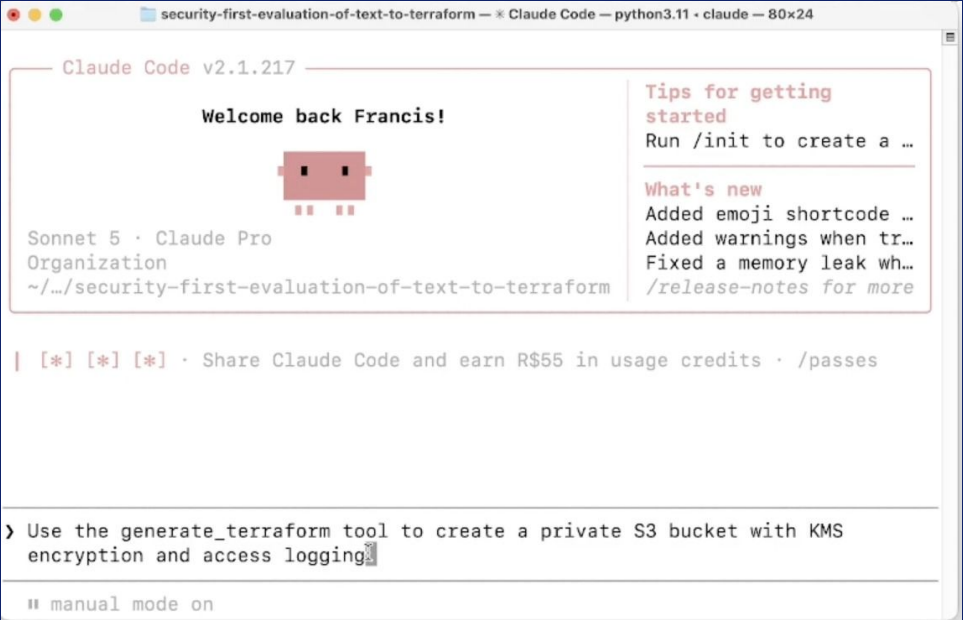


Modelo C4 (Container) — zonas Local, GitLab CI/CD e AWS Cloud. Nos próximos slides, cada zona em detalhe. Figura 2 do paper.

01 · SOLUÇÃO PROPOSTA

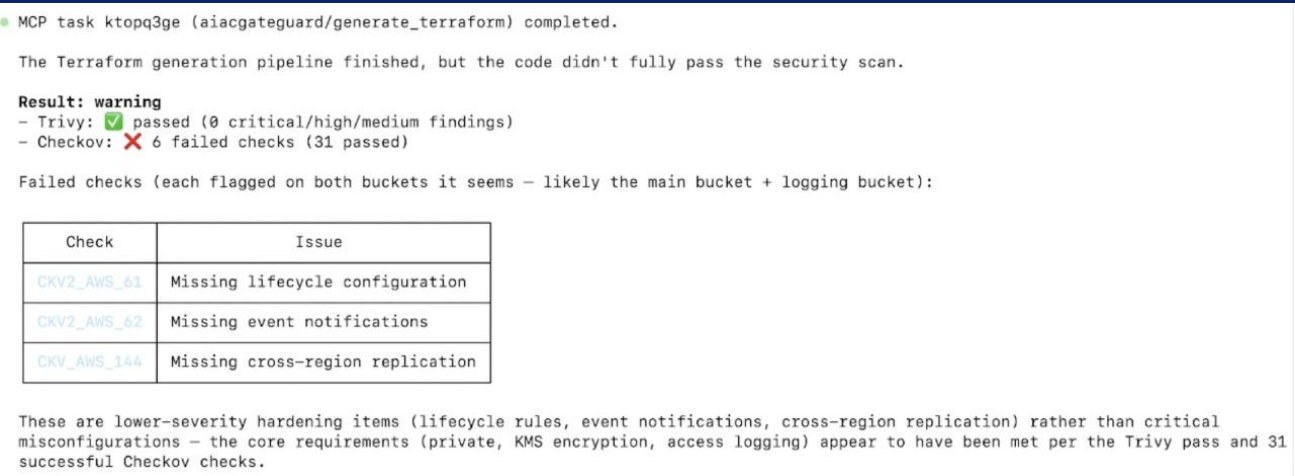
Prova real — MCP em ação

PROMPT · CLAUDE CODE



The screenshot shows the Claude Code v2.1.217 interface. It includes a welcome message "Welcome back Francis!" with a small robot icon. There are sections for "Tips for getting started" (Run /init to create a ...) and "What's new" (Added emoji shortcode, Added warnings when tr..., Fixed a memory leak wh..., /release-notes for more). A prompt at the bottom asks to "Use the generate_terraform tool to create a private S3 bucket with KMS encryption and access logging".

RESPOSTA NO CHAT · ACHADOS DOS SCANNERS



The screenshot shows the chat response to the prompt. It states that the MCP task ktopq3ge (aiacgateguard/generate_terraform) completed. The Terraform generation pipeline finished, but the code didn't fully pass the security scan. The result is a warning: Trivy passed (0 critical/high/medium findings) and Checkov failed 6 checks (31 passed). A table lists the failed checks: CKV2_AWS_61 (Missing lifecycle configuration), CKV2_AWS_62 (Missing event notifications), and CKV_AWS_144 (Missing cross-region replication). A note at the bottom states that these are lower-severity hardening items rather than critical misconfigurations.

Check	Issue
CKV2_AWS_61	Missing lifecycle configuration
CKV2_AWS_62	Missing event notifications
CKV_AWS_144	Missing cross-region replication

Capturas reais do vídeo de demonstração — Figura 3 do paper.

02

Avaliação experimental

Como validamos a proposta

Configuração experimental

Item	Configuração
Modelos	1 modelo (GPT-5.4)
Cenários	3 cenários Terraform AWS, com identificadores Checkov/Trivy esperados
Scanners	Checkov (conformidade de políticas) + Trivy (severidade de vulnerabilidades)
Parâmetros	Zero-shot, prompt L3 (Security detailed Level), 1 repetição

Reprodutibilidade

Código, prompts, cenários e vídeo no GitLab (open-source).



aponte a câmera · repositório no GitLab

02 · AVALIAÇÃO EXPERIMENTAL

Resultados

DEMONSTRAÇÃO · PIPELINE REDUZIDO VIA MCP (3 CENÁRIOS, 1 LLM, k=1)

Trivy

0 achados

Nenhuma vulnerabilidade de severidade crítica ou alta detectada.

Checkov

6 pendências de hardening

- Missing lifecycle configuration
- Missing event notifications
- Missing cross-region replication

CKV2_AWS_61 · CKV2_AWS_62 · CKV_AWS_144

Status	Pipeline	Created by	Stages	Actions
✓ Passed 00:14:34 2 hours ago	#2704127824 fix: server mcp feat/sf-sbseg 0040df2b latest branch api		» ✓ ✓ ✓ » ✓ ✓ » ✓ » »	▶ ▼ 📄 ▼

Demonstração com pipeline reduzido — o estudo completo (7 modelos × 17 cenários) está no paper complementar.

02 · AVALIAÇÃO EXPERIMENTAL

Resultados — 3 cenários (gpt-5.4)

DEMONSTRAÇÃO AMPLIADA · MESMO PIPELINE VIA MCP (gpt-5.4, k=1, 3 CENÁRIOS)

CENÁRIO (gpt-5.4 · k=1)	TERRAFORM FMT	VALIDATE	CHECKOV	TRIVY	OVERALL SCORE
S3 + KMS + logging create..._apply_kms_server_side			 4 pendências (17/21 checks)	 0 achados (crít./alto/méd.)	 com ressalvas 80
S3 nomeado (“my_bucket”) create..._named_my_bucket			 7 pendências (6/13 checks)	 5 high + 1 medium	 reprovado 47
VPC + CIDR create..._vpc_resource			 bloqueado no Estágio 2	 bloqueado no Estágio 2	 reprovado 50
Aprovado Pendência encontrada Bloqueado no estágio anterior					

Mesma demonstração da página anterior, agora nos 3 cenários do benchmark_test.yaml — sintaxe válida (fmt/validate) não garante conformidade de segurança.

Dados do benchmark onde a solução foi aplicada

7

Modelos avaliados

3 LLMs (API) + 4
SLMs (inferência
local), no mesmo
pipeline.

17

Cenários AWS

Terraform cobrindo
múltiplos recursos e
níveis de
especificidade.

3.570

Avaliações

Sintaxe e segurança
se mostraram
propriedades
amplamente
independentes.

O que os resultados sustentam

- Validade sintática e conformidade de segurança são propriedades amplamente independentes.
- O pipeline automatiza geração, validação, varredura, avaliação e relatório sem alterar código.
- A extensão a novos modelos, cenários e scanners é puramente declarativa (YAML).

Ameaças à validade

- Cobertura limitada a Terraform e AWS; outras linguagens exigem novos adaptadores.
- Estágio 6 (Revise) faz uma rodada por vez; múltiplas rodadas exigem novo disparo manual.
- Testes estatísticos (Wilson, Bonferroni) rodam como pós-processamento, fora do Estágio 5.

03

Conclusões

O que fica e para onde vamos

Considerações finais

01

Contribuição

Pipeline security-first
com dual-scanner e
MCP
conversacional.

02

Evidência

7 modelos, 17
cenários, 3.570
avaliações:
segurança ≠ sintaxe.

03

Impacto

DevSecOps corrige
gaps antes da
produção; uso
educacional.

Trabalhos futuros

- Dashboard web para análise interativa de conformidade.
- Suporte plugável a CloudFormation, Pulumi, Ansible e OPA/Rego.
- Revisão automática multi-rodada, preservando o humano no loop por padrão.
- API REST multiusuário e integração dos testes estatísticos ao Estágio 5.

Colabore com o AI Horizon Labs

IA aplicada à Engenharia de Software, Cibersegurança e IaC.



ai-horizon-labs.github.io

03 · CONCLUSÕES

O artefato recebeu os quatro selos do CTA/SBSeg 2026



Obrigado!

Perguntas são bem-vindas

AUTOR DE CONTATO

Francis Luis Santos Vargas

francisvargas.aluno@unipampa.edu.br

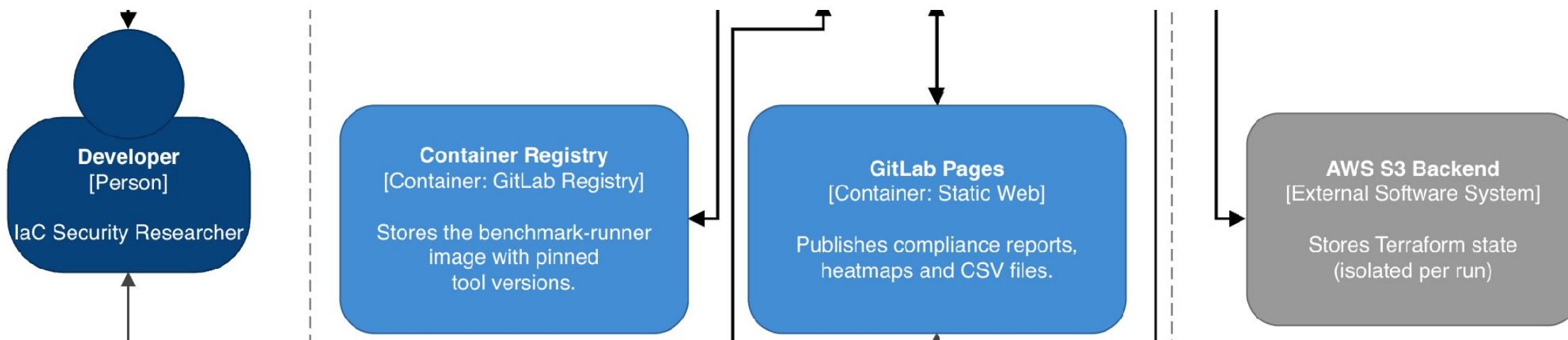
AI Horizon Labs · UNIPAMPA · PPGES



01 · SOLUÇÃO PROPOSTA

Onde os artefatos ficam

Visão geral → Aciona a geração → **Guarda os artefatos** → Extensões opcionais

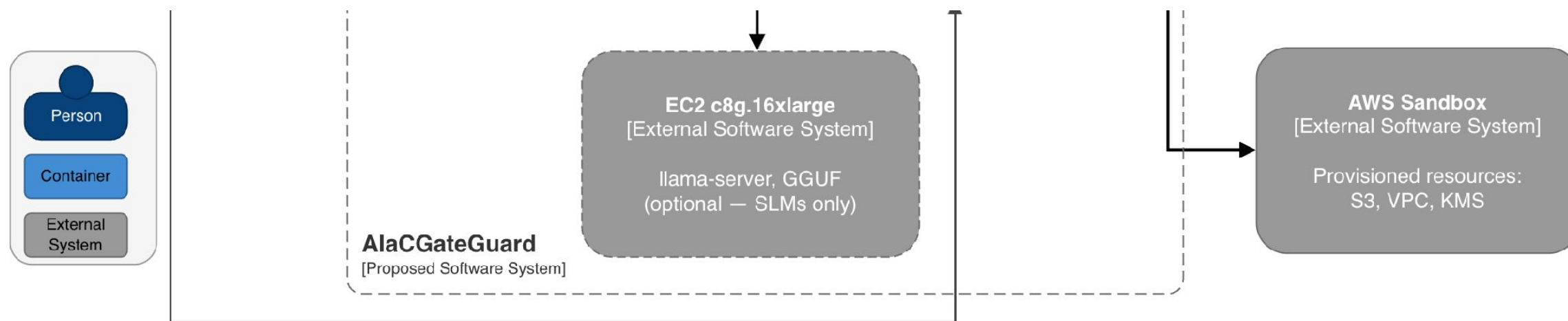


Zoom 2 de 3 — Container Registry guarda a imagem do runner; GitLab Pages publica relatórios e heatmaps; AWS S3 Backend guarda o state isolado por execução.

01 · SOLUÇÃO PROPOSTA

Extensões de infraestrutura opcionais

Visão geral → Aciona a geração → Guarda os artefatos → **Extensões opcionais**

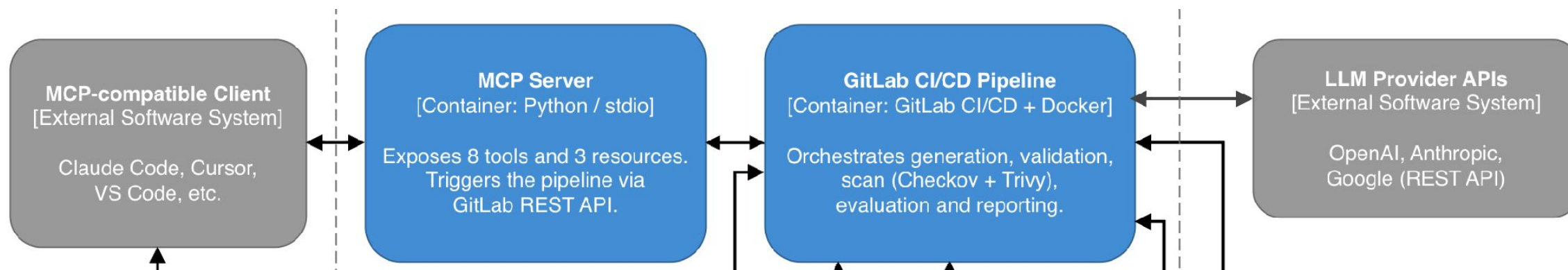


Zoom 3 de 3 — extensões opcionais: EC2 para SLMs locais (llama-server/GGUF) e AWS Sandbox para infraestrutura real provisionada sob demanda.

01 · SOLUÇÃO PROPOSTA

Como o pipeline é acionado

Visão geral → **Aciona a geração** → Guarda os artefatos → Extensões opcionais



Zoom 1 de 3 — o cliente MCP aciona o MCP Server, que dispara o pipeline GitLab CI/CD; o pipeline chama as LLM Provider APIs para gerar o Terraform.